SOLUÇÕES MOBILE

prof. Mariane J Melo





React Native:
reta final

Persistência de Dados

Além de toda a interatividade que um aplicativo deve fornecer ao usuário, em boa parte das situações alguns dados deverão "durar" além do uso atual do app. E assim como em um app Nativo (android ou iOS), em um app feito em ***React Native*** não poderia ser diferente.

Uma lista de preferências do usuário, uma configuração a ser salva, registros do usuário logado no momento e etc. Uma hora ou outra você como desenvolvedor terá que garantir que "coisas" permaneçam "vivas" mesmo depois da aplicação estar "morta".

Persistência de Dados

Em **React Native *com Expo)*** temos algumas soluções disponíveis para trabalho:

- AsyncStorage
- expo-sqlite
- Firebase



A persistência mais básica

AsyncStorage

AsyncStorage

Em uma analogia direta ao *Android Nativo*, o pacote **AsyncStorage** seria o equivalente ao que conhecemos por ***SharedPreferences***. Esta lib permite salvar e recuperar pares de chave/valor de tipos de dados do tipo string. Estes dados irão persistir na sessão do usuário (mesmo que sua aplicação seja morta).

Este tipo de armazenamento seria o caminho ideal para salvar pequenos dados e configurações que não precisam ser sincronizadas na nuvem e pertence apenas à instalação atual do app. Por exemplo, os dados do usuário logado no momento em seu app, poderiam ser salvos de alguma forma com **AsyncStorage**, bem como as preferências dele em relação ao "tema escuro ou claro" do aplicativo.

Instalando o AsyncStorage

Sendo uma lib externa (que não vem de fábrica habilitada) o pacote `**@react-native-community/async-storage**` necessita ser instalado em nosso projeto sempre que pretendermos usar o mesmo para alguma necessidade.

Instalando no **Expo**:

```
npx expo install @react-native-async-storage/async-storage
```

Utilizando AsyncStorage

O uso do ***AsyncStorage*** é bem simples, basta importar a lib usando a seguinte linha no começo de seu arquivo:

```
import AsyncStorage from '@react-native-async-storage/async-storage';
```

Depois temos dois métodos principais para trabalho: **`setItem`** e **`getItem`**:

https://docs.expo.dev/versions/latest/sdk/async-storage/

Utilizando AsyncStorage

Para **salvar um item** em nosso projeto, podemos seguir este caminho:

```
const salvarNome = async (value) => {
  try {
    await AsyncStorage.setItem('@nomePessoa', value)
  } catch (e) {
    // saving error
  }
}
```

Utilizando AsyncStorage

Caso seja necessário salvar um objeto complexo, composto de várias propriedades, teremos que dar um jeito de "transformá-lo" em string. O jeito mais fácil de fazer isso com JavaScript é converter o objeto para o formato JSON antes de salvá-lo.

```
const salvarPessoa = async (pessoa) => {
  try {
    const jsonValue = JSON.stringify(pessoa)
    await AsyncStorage.setItem('@pessoa', jsonValue)
  } catch (e) {
    // saving error
  }
}
```

Utilizando AsyncStorage

Após salvar um item com sucesso, em um segundo momento quando necessitarmos **recuperar** o valor (por exemplo, depois de reiniciar o aplicativo) podemos usar o método getItem chamando o item pelo mesmo nome de chave usado para salvar:

```
const getNome = async () => {
  try {
    const value = await AsyncStorage.getItem('@nomePessoa')
    if(value !== null) {
      // se o value for diferente de null, quer dizer que já havia sido salvo anteriormente.
      return value;
    }
  } catch(e) {
    // error reading value
  }
}
```

Utilizando AsyncStorage

Caso o valor recuperado seja complexo (um objeto por exemplo) devemos lembrar que o mesmo foi salvo no formato JSON, para restaurá-lo ao formato de Objeto novamente, logo após o processamento da função getItem devemos usar o método **JSON.parse**:

```
const getPessoa = async () => {
  try {
    const jsonValue = await AsyncStorage.getItem('@pessoa')
    if(jsonValue !== null) {
      // se o jsonValue for diferente de null, quer dizer que já havia sido salvo anteriormente.
      return JSON.parse(jsonValue);
    }
  } catch(e) {
    // error reading value
  }
}
```

Utilizando AsyncStorage

Se porventura em algum momento você necessitar limpar ou remover o valor de uma chave, a lib disponibiliza um método chamado **removeItem**:

```
const removePessoa = async () => {
  try {
    await AsyncStorage.removeItem('@pessoa')
  } catch(e) {
    // remove error
  }

  console.log('Removido com Sucesso')
}
```



Banco de Dados *"de Verdade"*

expo-sqlite

SQLite

Por ser altamente portável, autocontido e extremamente leve, o **SQLite** se tornou a solução nativa de banco de dados para muitas plataformas móveis como o iOS e também o Android. Sendo uma biblioteca, o **SQLite** lê e escreve diretamente no arquivo de banco de dados no *storage* do aparelho.

O uso do **SQLite** é recomendado onde a simplicidade da administração, implementação e manutenção são mais importantes que incontáveis recursos que bancos de dados tradicionais, mais voltados para aplicações complexas, possivelmente implementam *(ex: Postgres ou Oracle)*.

SQLite

No caso do ***React Native***, as duas alternativas mais populares de adicionar recursos de *SQLite* em nossa aplicação são as bibliotecas **expo-sqlite** e **react-native-sqlite-storage**.

Vamos prosseguir nossos estudos focando no pacote **expo-sqlite**, que diferente de sua nomenclatura *não funciona apenas em Expo*.



Instalando o expo-sqlite

Por ser uma lib original do mesmo time responsável pelo core do Expo, o processo de instalação nesta plataforma será bem fácil:

```
npx expo install expo-sqlite
```

Usando o expo-sqlite

Para iniciarmos o uso do *expo-sqlite* em nosso projeto devemos começar importando a lib e seus métodos no começo de nosso arquivo:

```
import * as SQLite from 'expo-sqlite';
```

Após importado, devemos lembrar que estamos trabalhando com um banco de dados tradicional baseado em SQL Ansi, em alguns momentos teremos que escrever nossas **querys na unha**.

Uma das querys mais importantes para iniciarmos nosso banco são as querys relacionadas com **CREATE TABLE** usadas para criar todas as tabelas de banco de dados que poderão ser usadas em nosso aplicativo.

Usando o expo-sqlite

Primeiro podemos criar uma variável do tipo string template (usando aspas invertidas ``) contendo o SQL básico de criação para nossa primeira tabela:

```
const SQL_CREATE = `
CREATE TABLE IF NOT EXISTS produtos (
id INTEGER PRIMARY KEY autoincrement,
name varchar(255) NOT NULL,
valor real NOT NULL
)
`;
```

Usando o expo-sqlite

Na sequência devemos abrir nosso banco dando um nome para o arquivo .sqlite que será salvo no disco do aparelho do usuário:

```
const db = SQLite.openDatabaseSync("exemploApp.sqlite");
```

Eu gosto de pensar sempre que no momento que eu "aceito" que meu app vai precisar de um banco de dados, diversas telas podem precisar usar o mesmo, por tanto eu costumo criar um arquivo chamado "db.js" na raiz do meu projeto a fim de facilitar o acesso a essa database que devemos conectar sempre que precisar fazer um SELECT ou INSERT.

Usando o expo-sqlite

Neste arquivo ***db.js*** eu costumo fazer algo assim:

```
import * as SQLite from 'expo-sqlite';

const DATABASE_NAME = "exemploApp.sqlite";
const SQL_CREATE = `...`;

let db = null;

export default function openDB() {
  if(!db) {
    db = SQLite.openDatabaseSync(DATABASE_NAME);

    // primeira vez que iremos abrir a conexão,
    // tentaremos criar nossas tabelas
    db.withTransactionSync(() => {
      db.execSync(SQL_CREATE);
    });
  }

  return db;
}
```

Usando o expo-sqlite

Assim no futuro sempre que eu precisar usar o banco de dados de qualquer tela eu posso importar o arquivo "db.js":

```
import openDB from "./db";

const db = openDB();

function MinhaTela() {

    useEffect(() => {
        const rows = db.getAllSync("select * from usuarios", []);
        console.log(JSON.stringify(rows));
    }, []);

  return ...;
}
```

Usando o expo-sqlite

Voltando ao funcionamento do **expo-sqlite**, após abrirmos nossa conexão com o banco *(usando openDatabaseSync)* teremos a nossa disposição diversos métodos para inserir dados ou realizar consultas, sempre usando SQL.

- db.execAsync ou db.execSync
- db.runAsync ou db.runSync
- db. getFirstAsync..
- db. getAllAsync..

https://docs.expo.dev/versions/latest/sdk/sqlite/

Usando o expo-sqlite

O método `db.**withTransactionSync**` inicia uma "transação" em nosso banco imbuída de uma proteção relacionada a "auto-rollback".

Ao iniciar uma nova transação você poderá executar uma série de comandos SQL e caso algum comando de erro, a transação será revertida e o estado do banco voltará para antes de você ter começado a operar na transação.



```
Promise.all([
  // 1. A new transaction begins
  db.withTransactionAsync(async () => {
    // 2. The value "first" is inserted into the test table and we wait 2
    //    seconds
    await db.execAsync('INSERT INTO test (data) VALUES ("first")');
    await sleep(2000);

    // 4. Two seconds in, we read the latest data from the table
    const row = await db.getFirstAsync<{ data: string }>('SELECT data FROM test');

    // ❌ The data in the table will be "second" and this expectation will fail.
    //    Additionally, this expectation will throw an error and roll back the
    //    transaction, including the `UPDATE` query below since it ran within
    //    the transaction.
    expect(row.data).toBe('first');
  }),
  // 3. One second in, the data in the test table is updated to be "second".
  //    This `UPDATE` query runs in the transaction even though its code is
  //    outside of it because the transaction happens to be active at the time
  //    this query runs.
  sleep(1000).then(async () => db.execAsync('UPDATE test SET data = "second"')),
]);
```

Usando o expo-sqlite

O db disponibiliza os métodos ***`execAsync, runAsync, getAllAsync`*** e seus variantes SYNC, e estes são basicamente os métodos que precisamos. Esses métodos recebem os seguintes argumentos nesta ordem de preenchimento:

1. **sqlStatement**: seu código de query SQL-ansi
2. **arguments**: argumentos para preencher uma query "preparada"

Vamos ver os retornos desses métodos...



Usando o expo-sqlite

O método execAsync não possui nenhum retorno. Usamos quando queremos executar algum SQL sem necessidade de avaliar o retorno da operação.

```
// `execAsync()` is useful for bulk queries when you want to execute altogether.
// Please note that `execAsync()` does not escape parameters and may lead to SQL injection.
await db.execAsync(`
PRAGMA journal_mode = WAL;
CREATE TABLE IF NOT EXISTS test (id INTEGER PRIMARY KEY NOT NULL, value TEXT NOT NULL, intValue INTEGER);
INSERT INTO test (value, intValue) VALUES ('test1', 123);
INSERT INTO test (value, intValue) VALUES ('test2', 456);
INSERT INTO test (value, intValue) VALUES ('test3', 789);
`);
```



Usando o expo-sqlite

Com o método runAsync podemos pegar, por exemplo, o id do item inserido.

```
// `runAsync()` is useful when you want to execute some write operations.
const result = await db.runAsync('INSERT INTO test (value, intValue) VALUES (?, ?)', 'aaa', 100);
console.log(result.lastInsertRowId, result.changes);
await db.runAsync('UPDATE test SET intValue = ? WHERE value = ?', 999, 'aaa'); // Binding unnamed parameters from variadic arguments
await db.runAsync('UPDATE test SET intValue = ? WHERE value = ?', [999, 'aaa']); // Binding unnamed parameters from array
await db.runAsync('DELETE FROM test WHERE value = $value', { $value: 'aaa' }); // Binding named parameters from object
```

Usando o expo-sqlite

Com o método getAllAsync podemos pegar o retorno de uma consulta

```
// `getAllAsync()` is useful when you want to get all results as an array of objects.
const allRows = await db.getAllAsync('SELECT * FROM test');
for (const row of allRows) {
  console.log(row.id, row.value, row.intValue);
}
```



Usando o expo-sqlite

Óbvio que dentro do React Native, devemos seguir algumas normas do ecossistema React, como o uso de ***useState***, ***useEffect*** e outros hooks que irão ditar o estado dos nossos dados e também em que momento nossos códigos serão executados. Nos próximos slides, um exemplo de uso com uma tela de produtos que puxa o nome e preço de produtos salvos no banco.

Usando o expo-sqlite

```
import {View, Text, TextInput, Button} from 'react-native';
import {useState, useEffect} from 'react'

import openDB from '../database/db';

const db = openDB();

export default function Produtos (){

  const [produtos, setProdutos] = useState([]);
  const [nome, setNome] = useState('');
  const [valor, setValor] = useState('');
  const [editaId, setEditaId] = useState(null)
```

```
const adicionarProdutos = async () => {
  try {
    await db.runAsync(
      'INSERT INTO produtos (name, valor) VALUES (?, ?)',
      [nome, parseFloat(valor)]
    );

    alert('adicionado');
    setNome('')
    setValor('')

    carregarProdutos();
  } catch (e) {
    console.log(e);
  }
};
```

Usando o expo-sqlite

```
const carregarProdutos = async () => {
  try {
    const rows = await db.getAllAsync("select * from produtos");
    setProdutos(rows);
    console.log(rows);
  } catch (e) {
    console.log(e);
  }
};
```

Usando o expo-sqlite

```
const deletarProdutos = async (id) => {
  try {
    await db.runAsync (
      'Delete from produtos where id = ?', [id]);
    carregarProdutos();

  }
  catch(e) {
    console.log(e)
  }
}
```

Usando o expo-sqlite

```
const atualizarProduto = async () => {
  try{
      db.runAsync(
      'UPDATE produtos SET name = ?, valor = ? WHERE id = ?',
      [nome, parseFloat(valor), editaId]
  );
  setEditaId(null);
  setNome('')
  setValor('')
  await carregarProdutos(); // para esperar o update terminar e dps mostrar
  }
  catch(e) {
    console.log(e)
  }

};
  const editarProduto = (a) => {
  setNome(a.name);
  setValor(a.valor.toString());
  setEditaId(a.id);
};

useEffect(() => {
  carregarProdutos()
}, []);
```

Usando o expo-sqlite

```
return(
  <View>
    <TextInput placeholder='nome' value={nome} onChangeText={setNome}/>
    <TextInput placeholder='valor' value={valor} onChangeText={setValor}/>

    {editaId ? (
      <Button title="Atualizar Produto" onPress={atualizarProduto} />
    ) : (
      <Button title="cadastrar Produto" onPress={adicionarProdutos} />
    )}

    <Text> PRODUTOS </Text>
    {produtos.map((a) => (
      <View key = {a.id}>
        <Text>{a.name}</Text>
        <Text>{a.valor}</Text>
        <Button  title='deletar produto' onPress={() => deletarProdutos(a.id)}/>
        <Button title="editar Produto" onPress={() => editarProduto(a)} />

      </View>
    ))}
  </View>
)
}
```



Pedido permissão

Permissions

Por que pedir permissões?

Um dos pontos de projeto centrais da arquitetura de segurança do **SO Android** é que, por padrão, nenhum app tem permissão de realizar nenhuma operação que prejudique outros apps, o sistema operacional ou o usuário.

Isso abrange a leitura e a gravação de dados privados do usuário (como contatos ou e-mails), leitura ou gravação dos arquivos, realização de acesso de rede (internet), manter a tela do dispositivo ativo etc.

Apps para Android e IOS precisam solicitar permissão para acessar dados confidenciais do usuário (como contatos e SMS), bem como recursos específicos do sistema (como câmera, galeria de fotos e Internet). Dependendo do recurso, o sistema pode conceder a permissão automaticamente ou pedir ao usuário que aprove a solicitação.

Permissions

Usando **Expo** o módulo que era utilizado era o `***expo-permissions***` que deve ser instalado assim:

```
npx expo install expo-permissions
```

Porém, expo-permissions está obsoleto, no Expo, devemos usa permissão direto no módulo específico

Permissions

Porém, no **EXPO** atualmente *(e principalmente)* cada módulo chave possui seu método exclusivo para solicitar as permissões adequadas para seu funcionamento e na atual situação acabamos por usar o módulo ***`expo-permissions`*** apenas quando não estamos usando módulos fornecidos pela documentação do Expo.

Por exemplo o módulo de Câmera do Expo possui uma função exclusiva para solicitar as permissões necessárias para podermos usar a câmera de um aparelho:

```
npx expo install expo-camera
```

```
const [permission, requestPermission] =Camera.useCameraPermissions();
```

Permissions

```jsx
import { Camera } from 'expo-camera';
import { useState, useEffect } from 'react';
import { View, Text, Button } from 'react-native';

export default function App() {
  const [permissao, setPermissao] = useState(null);

  useEffect(() => {
    pedirPermissao();
  }, []);

  const pedirPermissao = async () => {
    const { status } = await Camera.requestCameraPermissionsAsync();
    setPermissao(status === 'granted');
  };

  if (permissao === null) return <Text>Pedindo permissão...</Text>;
  if (permissao === false) return <Text>Sem acesso à câmera</Text>;

  return (
    <View>
      <Text>Câmera liberada!</Text>
    </View>
  );
}
```

Permissions

Assim como o módulo **ImagePicker**:

```
ImagePicker.requestCameraPermissionsAsync()
ImagePicker.requestMediaLibraryPermissionsAsync()
```

E outro exemplo o módulo de *Localização* (**expo-location**):

```
let { status } = await Location.requestForegroundPermissionsAsync();
```

Permissions

No caso de um aplicativo **React Native padrão/puro** *(sem expo)*, obrigatoriamente um app precisa divulgar as permissões necessárias incluindo tags `<uses-permission>` no arquivo manifesto do app.

Por exemplo um aplicativo que precise enviar mensagens SMS de maneira programática teria que adicionar a seguinte linha em seu ***AndroidManifest.xml***:

```
<uses-permission android:name="android.permission.SEND_SMS" />
```

Recomendo consultar a documentação oficial do módulo a ser usado junto da configuração acima:

https://reactnative.dev/docs/permissionsandroid



OBRIGADO